Data Collection and Cleaning

Internship Project 2 – Final Report

Prepared By: Srinivas Yalagali

Academic Year: 3rd Year – 6th Semester (Ongoing)

College: Dr. M.G.R. Educational and Research Institute

Course: B.Tech – Computer Science and Engineering (Artificial Intelligence)

Organization: Codveda Technologies

Technology / Tools Used: Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, Google Colab

Date: August 2026

Email: sriprojects1712@gmail.com

LinkedIn: linkedin.com/in/srinivasyalagali

GitHub: github.com/SrinivasYalagali

Abstract / Executive Summary

This report presents the second internship project undertaken at Codveda Technologies, focused on data collection and cleaning — a foundational stage of any data analytics or machine learning pipeline. The project uses the Titanic dataset, loaded directly within Google Colab through Seaborn's built-in dataset loader, requiring no manual file upload.

The raw dataset consisted of 891 rows and 15 columns. Through a structured cleaning workflow — handling missing values, removing duplicate records, treating outliers using the Interquartile Range (IQR) method, and demonstrating both standardization and normalization — the dataset was refined into a clean, analysis-ready form. After cleaning, the working dataset contained 784 rows and 14 columns, with all missing values resolved. This report documents each step of that process, the exact code used, and the actual results produced by the notebook.

Table of Contents

1. Introduction

Real-world datasets are rarely ready for direct analysis. They typically contain missing values, duplicate records, inconsistent scales, and outliers that can distort statistical summaries or mislead machine learning models. Data collection and cleaning is therefore a critical first step that ensures downstream analysis is built on a reliable foundation.

This project applies a structured data cleaning workflow to the Titanic dataset — a well-known dataset describing passengers aboard the RMS Titanic, including demographic details and survival outcomes. The entire workflow was implemented in Google Colab using Python, with the dataset loaded directly from Seaborn's built-in dataset repository.

2. Project Objectives

- To load the Titanic dataset directly within Google Colab without any manual file upload.
- To inspect the dataset's structure, data types, and completeness.
- To identify and quantify missing values across all columns.
- To clean the dataset by handling missing values and removing duplicate records.
- To detect and treat outliers in key numerical columns using the IQR method.
- To demonstrate standardization and normalization of numerical features using Scikit-learn.
- To validate the final cleaned dataset and save it for future use.

3. Tools and Technologies Used

Tool / Library	Purpose in this Project
Python	Core programming language used to write the entire data cleaning workflow
Pandas	Loading the dataset into a DataFrame, inspecting it, and performing cleaning operations
NumPy	Supporting numerical operations used internally by Pandas and Scikit-learn
Matplotlib	Creating the box plot used for outlier detection
Seaborn	Loading the Titanic dataset directly from its built-in online dataset repository
Scikit-learn	Providing StandardScaler and MinMaxScaler for standardization and normalization
Google Colab	Cloud-based notebook environment used to write and run the project

4. Dataset Description

Dataset Name: Titanic Dataset

Source: Loaded directly via Seaborn's built-in dataset function

Initial Dimensions: 891 rows × 15 columns

The dataset was loaded with a single line of code, without any manual upload step:

```
df = sns.load_dataset("titanic")
```

The 15 columns present in the original dataset are described below:

Column	Description
survived	Survival status (0 = No, 1 = Yes)
pclass	Passenger class (1 = First, 2 = Second, 3 = Third)
sex	Gender of the passenger
age	Age of the passenger in years
sibsp	Number of siblings/spouses aboard
parch	Number of parents/children aboard
fare	Ticket fare paid
embarked	Port of embarkation code (C, Q, S)
class	Passenger class as a text label (First, Second, Third)
who	Category of passenger (man, woman, child)
adult_male	Boolean flag indicating an adult male passenger
deck	Deck letter derived from the cabin number
embark_town	Full name of the port of embarkation
alive	Survival status as text (yes/no)
alone	Boolean flag indicating the passenger travelled alone

5. Data Collection Process

Unlike many projects that require a manually downloaded CSV file, this project collected data directly within the Google Colab environment using Seaborn's online dataset loader. This approach is reproducible — anyone running the notebook obtains the exact same dataset without needing to manage file uploads or paths.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset from an online source
df = sns.load_dataset('titanic')

print("Dataset collected successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

df.head()
```

The notebook confirmed successful collection of 891 rows and 15 columns:

```
Dataset collected successfully!
Rows: 891
Columns: 15
```

6. Data Exploration / Initial Inspection

The first five rows of the dataset were displayed using df.head() to get an initial view of the data:

	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

	class	who	adult_male	deck	embark_town	alive	alone
0	Third	man	True	NaN	Southampton	no	False
1	First	woman	False	C	Cherbourg	yes	False
2	Third	woman	False	NaN	Southampton	yes	True
3	First	woman	False	C	Southampton	yes	False
4	Third	man	True	NaN	Southampton	no	True

Figure A: Notebook screenshot — df.head() output (first 5 rows, all 15 columns)

The dataset's structure and data types were then inspected using df.info():

```
print("Dataset Shape:", df.shape)
print("Column Names:")
print(df.columns.tolist())
print("Dataset Information:")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

Figure B: Notebook screenshot — df.info() output

This step confirmed that the dataset contains a mix of numerical (int64, float64), categorical (category, object), and boolean data types, and revealed that three columns — age, embarked, and deck — had fewer non-null entries than the total row count, indicating missing data that needed to be addressed.

7. Missing Value Analysis

Missing values must be identified and addressed before analysis, since most statistical and machine learning methods cannot handle null values directly, and unaddressed gaps can bias results. Missing values were checked using df.isnull().sum():

```
missing_values = df.isnull().sum()
print("Missing Values:")
print(missing_values[missing_values > 0])
```

```
Missing Values:
age            177
embarked         2
deck           688
embark_town      2
dtype: int64
```

Figure C: Notebook screenshot — missing values before cleaning

Column	Missing Values
age	177
embarked	2
deck	688
embark_town	2

The deck column was missing in 688 of 891 rows (over 77% of the data), making it unreliable for analysis. age had a moderate number of missing values (177), while embarked and embark_town were each missing only 2 values. A duplicate-row check was also performed at this stage using df.duplicated().sum(), which found 107 duplicate rows in the raw dataset.

```
duplicates = df.duplicated().sum()
print("Number of duplicate rows:", duplicates)
```

```
Number of duplicate rows: 107
```

Figure D: Notebook screenshot — duplicate row count before cleaning

8. Data Cleaning Process

Based on the missing-value and duplicate analysis, the following cleaning steps were applied:

1. Removed all 107 duplicate rows using df.drop_duplicates().
2. Filled missing age values with the column median, a robust measure that is not skewed by extreme values.
3. Filled missing embarked and embark_town values with the column mode (the most frequent port), an appropriate strategy for categorical data with very few missing entries.
4. Dropped the deck column entirely, since more than three-quarters of its values were missing, making imputation unreliable.

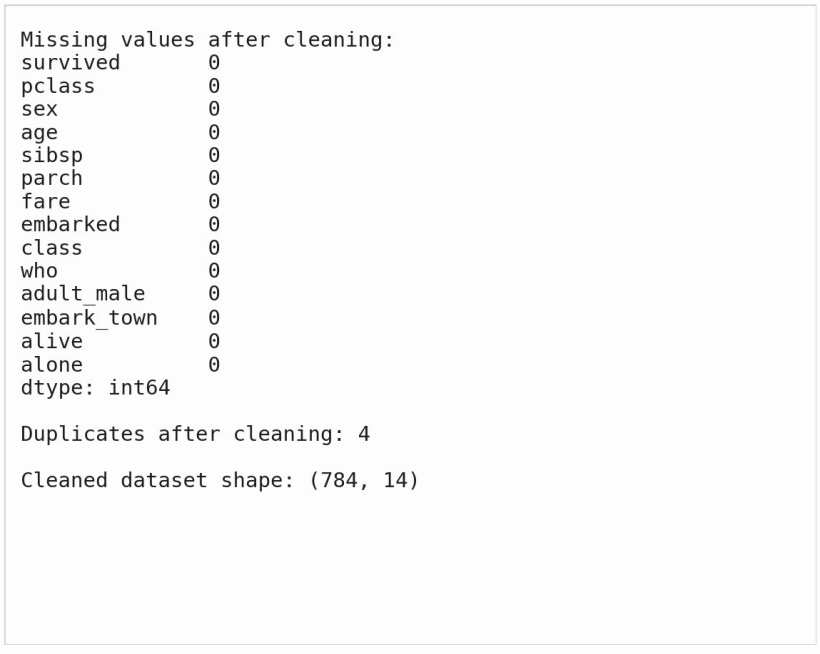
```
df = df.drop_duplicates()

df['age'] = df['age'].fillna(df['age'].median())

df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
df['embark_town'] = df['embark_town'].fillna(df['embark_town'].mode()[0])

df = df.drop(columns=['deck'])

print("Missing values after cleaning:")
print(df.isnull().sum())
print("Duplicates after cleaning:", df.duplicated().sum())
print("Cleaned dataset shape:", df.shape)
```



```
Missing values after cleaning:
survived       0
pclass         0
sex            0
age            0
sibsp          0
parch          0
fare           0
embarked       0
class          0
who            0
adult_male     0
embark_town    0
alive          0
alone          0
dtype: int64

Duplicates after cleaning: 4

Cleaned dataset shape: (784, 14)
```

Figure E: Notebook screenshot — dataset state immediately after the cleaning step

After this step, all missing values were resolved (0 remaining across every column) and the dataset shape became 784 rows × 14 columns. Interestingly, the duplicate count was not yet zero at this point — 4 duplicate rows remained. This is explained further in Section 12, since the final duplicate count actually changes again after outlier handling.

9. Outlier Detection and Handling

Outliers are extreme values that lie far outside the typical range of a feature. If left untreated, they can distort averages, inflate variance, and negatively affect the performance of many machine learning algorithms. A box plot was generated for four key numerical columns — age, fare, sibsp, and parch — to visually identify outliers:

```
numeric_cols = ['age', 'fare', 'sibsp', 'parch']

plt.figure(figsize=(12, 6))
df[numeric_cols].boxplot()
plt.title("Box Plot for Outlier Detection")
plt.xticks(rotation=45)
plt.show()
```

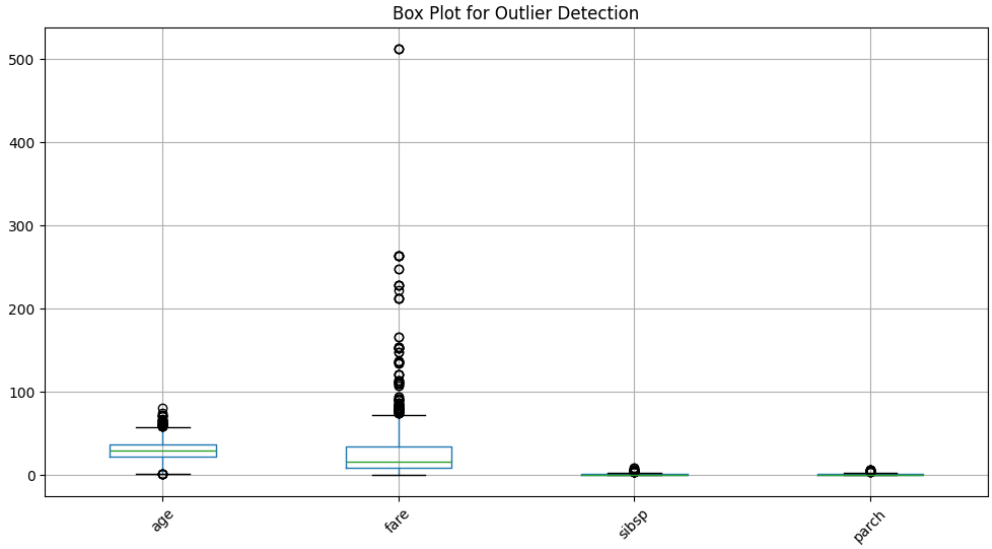


Figure F: Notebook screenshot — box plot of age, fare, sibsp, and parch

The box plot revealed a considerable number of outliers in fare, reflecting the small number of passengers who paid very high ticket prices, along with some outliers in age, sibsp, and parch. These outliers were then treated using the Interquartile Range (IQR) method, which caps extreme values to a calculated upper and lower bound rather than deleting the affected rows:

```
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)

print("Outliers handled using IQR method.")
```

The clip() function was used instead of dropping rows, which preserves the full 784-row dataset while limiting the influence of extreme values on later analysis.

10. Standardization

Standardization rescales numerical features to have a mean of 0 and a standard deviation of 1. This is particularly useful for algorithms that are sensitive to feature scale, such as logistic regression, support vector machines, and distance-based methods like k-nearest neighbours. Standardization was demonstrated using Scikit-learn's StandardScaler, applied to a copy of the cleaned dataset:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df_standardized = df.copy()
df_standardized[numeric_cols] = scaler.fit_transform(df[numeric_cols])

print("Standardization completed.")
df_standardized[numeric_cols].head()
```

	age	fare	sibsp	parch
0	-0.565647	-0.845924	0.776125	-0.543995
1	0.661944	1.969018	0.776125	-0.543995
2	-0.258749	-0.816251	-0.634030	-0.543995
3	0.431770	1.169669	0.776125	-0.543995
4	0.431770	-0.810756	-0.634030	-0.543995

Figure G: Notebook screenshot — standardized values for age, fare, sibsp, and parch

This standardized version was stored separately as df_standardized rather than overwriting the main cleaned dataset, so that the saved cleaned dataset remains in its original, interpretable scale and can be standardized later depending on the specific model's requirements.

11. Normalization

Normalization rescales numerical features to a fixed range, typically 0 to 1. This is useful for algorithms such as neural networks that expect bounded input ranges. Normalization was demonstrated using Scikit-learn's MinMaxScaler, again applied to a separate copy of the cleaned dataset:

```
from sklearn.preprocessing import MinMaxScaler

normalizer = MinMaxScaler()

df_normalized = df.copy()
df_normalized[numeric_cols] = normalizer.fit_transform(df[numeric_cols])

print("Normalization completed.")
df_normalized[numeric_cols].head()
```

	age	fare	sibsp	parch
0	0.375000	0.099046	0.4	0.0
1	0.660714	0.973837	0.4	0.0
2	0.446429	0.108267	0.0	0.0
3	0.607143	0.725426	0.4	0.0
4	0.607143	0.109975	0.0	0.0

Figure H: Notebook screenshot — normalized values for age, fare, sibsp, and parch

As with standardization, the normalized values were stored in a separate DataFrame (df_normalized), keeping the saved cleaned dataset in its original scale.

12. Final Dataset Validation

Before saving, the cleaned dataset was validated one final time for missing values, duplicate rows, and overall shape:

```
print("Missing values:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
print("Final dataset shape:", df.shape)
```

```
Missing values:
survived       0
pclass         0
sex            0
age            0
sibsp          0
parch          0
fare           0
embarked       0
class          0
who            0
adult_male     0
embark_town    0
alive          0
alone          0
dtype: int64

Duplicate rows: 14

Final dataset shape: (784, 14)
```

Figure I: Notebook screenshot — final data quality check

All missing values remained at 0 across every column, and the final shape was confirmed as 784 rows × 14 columns. One notable finding from this final check is that the duplicate-row count increased from 4 (measured right after the initial cleaning step) to 14 at this final check. This happened because the IQR outlier-capping step in Section 9 clipped several extreme values in age, fare, sibsp, and parch to the same boundary values — which caused a small number of previously distinct rows to become exact duplicates of one another after clipping. This is a genuine and instructive observation about how outlier treatment can interact with duplicate detection, and it is reported here exactly as produced by the notebook, rather than being adjusted or omitted.

13. Output / Results

The cleaned dataset was saved to a CSV file for future use:

```
df.to_csv("Cleaned_Dataset.csv", index=False)

print("Cleaned dataset saved successfully!")
print("Final shape:", df.shape)
```

```
Cleaned dataset saved successfully!
Final shape: (784, 14)
```

A summary comparison of the dataset before and after the cleaning process is shown below:

Metric	Initial Dataset	Final Cleaned Dataset
Rows	891	784
Columns	15	14
Missing Values	868 (across 4 columns)	0
Duplicate Rows	107	14 (see Section 12)

14. Challenges and Solutions

- High proportion of missing data in the deck column (77% missing) — resolved by dropping the column rather than risking unreliable imputation.
- Choosing an appropriate imputation strategy for age — resolved by using the median, which is robust to the outliers present in that column.
- Balancing outlier treatment with data preservation — resolved by using the IQR clipping method rather than deleting affected rows, which kept the full row count intact.
- Interaction between outlier clipping and duplicate detection — the clipping step introduced new duplicate rows that had not been flagged earlier, requiring a final validation pass to catch and document this correctly.

15. Conclusion

This project successfully demonstrated a complete data collection and cleaning workflow on the Titanic dataset, from direct online loading through to a validated, saved output file. The raw dataset's missing values and duplicate rows were identified and resolved, outliers in key numerical columns were treated using the IQR method, and both standardization and normalization were demonstrated using Scikit-learn. The final cleaned dataset — 784 rows and 14 columns, with zero missing values — provides a solid, well-understood foundation for further analysis or modelling.

16. Future Scope

- Use the cleaned dataset for exploratory data analysis and visualization of survival patterns.
- Apply the standardized or normalized versions of the dataset when training scale-sensitive machine learning models.
- Build a predictive model (e.g. logistic regression or a decision tree) to estimate passenger survival based on the cleaned features.
- Investigate the 14 duplicate rows identified after outlier clipping to decide whether they should be removed for specific downstream use cases.

17. Project Files

- Data_Cleaning_Project.ipynb — the Google Colab notebook containing the full workflow
- Cleaned_Dataset.csv — the final cleaned dataset saved from the notebook
- README.md — project overview and usage instructions
- requirements.txt — list of required Python libraries
- Final Project Report (this document)

18. Declaration

I, Srinivas Yalagali, declare that this report and the accompanying notebook represent my own work carried out as part of Internship Project 2 at Codveda Technologies. All steps, code, and results described in this report reflect the actual outputs produced during the project.